

ServiceAccount e Security Context Constraints (SCC)

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere il ruolo dei **ServiceAccount** come identità dei Pod in OpenShift
- Configurare SA custom con `automountServiceAccountToken: false` e `imagePullSecrets`
- Elencare le SCC predefinite di OpenShift e i loro livelli di privilegio
- Spiegare come OpenShift seleziona la SCC da applicare a un Pod
- Creare SCC custom per workload che richiedono permessi specifici
- Assegnare SCC a ServiceAccount tramite `oc adm policy add-scc-to-user`
- Diagnosticare errori di SCC su ARO e confrontare SCC con Pod Security Admission

Teoria e Concetti Chiave

ServiceAccount: Identità dei Pod

Ogni Pod in OpenShift viene eseguito sotto l'identità di un **ServiceAccount** (SA). Un SA è un account non-umano che:

- Appartiene a un namespace specifico
- Ha un token JWT montato automaticamente in ogni Pod
(`/var/run/secrets/kubernetes.io/serviceaccount/token`)
- Può avere **RoleBinding** e **ClusterRoleBinding** come qualsiasi utente
- Può avere **SCC** assegnate per controllare i privilegi del Pod

Caratteristica	Valore default
SA default del namespace	<code>default</code> (usato se non specificato)
Mount automatico token	<code>true</code> (da disabilitare se il Pod non chiama l'API)
Nome nel soggetto RBAC	<code>system:serviceaccount:<namespace>:<sa-name></code>

⚠ Attenzione: Non usare il ServiceAccount `default` per i Pod applicativi. Creare sempre un SA dedicato con i soli permessi necessari. Il SA `default` spesso ha SCC assegnate per altri scopi.

Auto-mount del Token e Best Practice

Il token montato automaticamente permette al processo nel container di chiamare l'API Kubernetes. Se il Pod non ne ha bisogno, il mount deve essere disabilitato:

```
# Disabilitare il mount automatico a livello di ServiceAccount
automountServiceAccountToken: false

# Oppure a livello di Pod (override del SA)
spec:
  automountServiceAccountToken: false
```

Quando disabilitare:

- Pod applicativi che non chiamano l'API Kubernetes
- Pod con accesso a dati sensibili (riduce la superficie di attacco)

Quando mantenere abilitato:

- Operator, controller e tool di CI/CD che interagiscono con l'API
- Pod che usano Workload Identity su ARO (token OIDC diverso dal token SA)

Security Context Constraints (SCC)

Le **SecurityContextConstraints** sono oggetti cluster-scoped esclusivi di OpenShift che controllano le azioni che un Pod può eseguire e le risorse che può accedere. Estendono e sostituiscono i Pod Security Standards di Kubernetes upstream.

SCC Predefinite in OpenShift

SCC	Privilegio	Uso tipico
<code>restricted-v2</code>	Minimo (default)	Workload applicativi standard
<code>nonroot-v2</code>	Basso	Processi non-root con UID fisso
<code>anyuid</code>	Medio	Container che richiedono un UID specifico (es. legacy apps)
<code>hostnetwork-v2</code>	Alto	Pod che usano il network namespace del nodo
<code>hostpath</code>	Alto	Pod che montano path arbitrari del filesystem del nodo
<code>node-exporter</code>	Alto	Monitoring agent sui nodi
<code>privileged</code>	Massimo	DaemonSet di sistema, driver del nodo

Campi Chiave di una SCC

Campo	Descrizione	Valore in restricted-v2
<code>allowPrivilegedContainer</code>	Permette privileged: true nel Security Context	<code>false</code>
<code>allowPrivilegeEscalation</code>	Permette processi che acquisiscono più privilegi del parent	<code>false</code>
<code>runAsUser.type</code>	Strategia UID: <code>MustRunAsRange</code> , <code>MustRunAs</code> , <code>RunAsAny</code>	<code>MustRunAsNonRoot</code>
<code>fsGroup.type</code>	Strategia <code>fsGroup</code>	<code>MustRunAs</code>
<code>seLinuxContext.type</code>	Strategia SELinux	<code>MustRunAs</code> (MCS automatico)
<code>volumes</code>	Tipi di volume consentiti	<code>configMap</code> , <code>downwardAPI</code> , <code>emptyDir</code> , <code>persistentVolumeClaim</code> , <code>projected</code> , <code>secret</code>
<code>allowedCapabilities</code>	Capabilities Linux aggiuntive consentite	<code>[]</code> (nessuna)
<code>requiredDropCapabilities</code>	Capabilities che devono essere droppate	<code>["ALL"]</code>

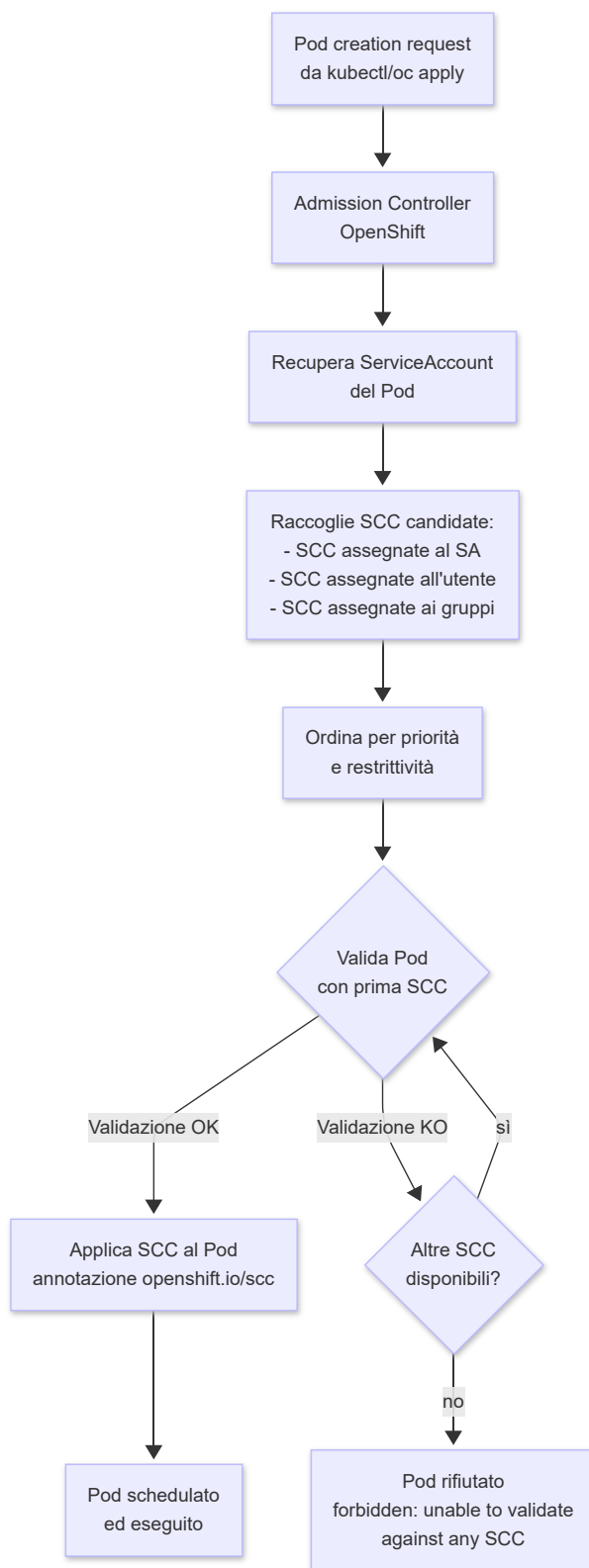
Algoritmo di Selezione della SCC

Quando un Pod viene creato, l'Admission Controller di OpenShift seleziona la SCC da applicare seguendo questo algoritmo:

1. **Raccoglie le SCC candidate:** tutte le SCC assegnate al ServiceAccount del Pod, all'utente che ha creato il Pod, e ai gruppi di cui fa parte
2. **Ordina le SCC per priorità:** prima per campo **priority** (numerico, più alto = prima), poi per restrittività (più restrittiva = prima)
3. **Valida il Pod con ogni SCC:** applica la prima SCC che valida con successo il security context del Pod
4. **Rifiuta il Pod:** se nessuna SCC valida il Pod, la creazione fallisce con **forbidden: unable to validate against any security context constraint**

Diagrammi

Flusso di Selezione SCC per un Pod



Comandi Pratici con Output Atteso

```
# Creare un ServiceAccount custom per un'applicazione
oc create serviceaccount myapp-sa -n myproject
```

```
# Output atteso:
# serviceaccount/myapp-sa created
```

```
# Elencare tutte le SCC disponibili nel cluster con il loro livello
oc get scc -o custom-columns=\
NAME:.metadata.name,\
PRIV:.allowPrivilegedContainer,\
RUNASUSER:.runAsUser.type,\
VOLUMES:.volumes
```

```
# Output atteso:
```

# NAME	PRIV	RUNASUSER	VOLUMES
# anyuid	false	RunAsAny	[...]
# hostnetwork-v2	false	MustRunAsRange	[...]
# node-exporter	false	RunAsAny	[*]
# nonroot-v2	false	MustRunAsNonRoot	[...]
# privileged	true	RunAsAny	[*]
# restricted-v2	false	MustRunAsNonRoot	[...]

```
# Verificare quale SCC è applicata a un Pod in esecuzione
oc get pod myapp-7f9d8b-abc12 -n myproject \
-o jsonpath='{.metadata.annotations.openshift\.io/scc}'
```

```
# Output atteso:
# restricted-v2
```

```
# Assegnare la SCC anyuid al ServiceAccount myapp-sa
oc adm policy add-scc-to-user anyuid \
system:serviceaccount:myproject:myapp-sa
```

```
# Output atteso:
# clusterrole.rbac.authorization.k8s.io/system:openshift:scc:anyuid added:
"system:serviceaccount:myproject:myapp-sa"
```

```
# Verificare le SCC assegnate a un ServiceAccount
oc adm policy who-can use scc anyuid
```

```
# Output atteso:
# Users:  system:serviceaccount:myproject:myapp-sa
# Groups: system:cluster-admins, system:masters
```

```
# Descrivere la SCC restricted-v2 per vedere tutti i vincoli
oc describe scc restricted-v2
```

```
# Output atteso (selezione):
# Name:                      restricted-v2
# Priority:                  <none>
# Access:
#   Users:                  <none>
#   Groups:                 system:authenticated
# Settings:
#   Allow Privileged:       false
#   Allow Privilege Escalation: false
#   Default Add Capabilities: <none>
#   Required Drop Capabilities: ALL
#   Allowed Capabilities:   <none>
#   Allowed Unsafe Sysctls: <none>
#   Run As User Strategy: MustRunAsNonRoot
```

```
# Diagnosticare un Pod che fallisce per SCC
oc describe pod failing-pod -n myproject | grep -A5 "Warning"
```

```
# Output atteso (Pod che richiede root):
# Warning Failed      2s      kubelet  Error: container has runAsNonRoot and image
will run as root
#
                                     (pod: "failing-pod", container: app)
```

```
# Verificare i permessi SCC dell'utente corrente
oc adm policy scc-subject-review \
  -f deployment.yaml \
  -n myproject
```

```
# Output atteso:
# RESOURCE      ALLOWED BY
# Pod/myapp     restricted-v2
```

Esempi YAML Completi

ServiceAccount Custom + Deployment con SA Dedicato

```

# ServiceAccount dedicato per l'applicazione
# Principio del minimo privilegio: SA senza permessi extra
apiVersion: v1
kind: ServiceAccount
metadata:
  name: myapp-sa
  namespace: myproject
  labels:
    app: myapp
  annotations:
    description: "ServiceAccount per l'applicazione myapp"
automountServiceAccountToken: false # ← disabilita il mount automatico del token
API
# (myapp non ha bisogno di chiamare l'API
Kubernetes)
---
# Deployment che usa il ServiceAccount dedicato
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: myproject
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      serviceAccountName: myapp-sa # ← usa il SA dedicato invece del SA
"default"
      automountServiceAccountToken: false # ← override a livello Pod (ridondante
ma esplicito)
      containers:
        - name: app
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          command: ["/bin/sh", "-c", "sleep 3600"]
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "500m"

```

SCC Custom per Applicazione Legacy (richiede UID specifico)


```

# SCC custom per applicazione legacy che deve girare con UID 1001
# Più permissiva di restricted-v2 ma meno di anyuid
apiVersion: security.openshift.io/v1
kind: SecurityContextConstraints
metadata:
  name: legacy-app-scc
  annotations:
    kubernetes.io/description: "SCC per applicazioni legacy che richiedono UID
1001"
# Utenti e gruppi che possono usare questa SCC
users: [] # ← popolato da oc adm policy add-scc-to-user
groups: [] # ← popolato da oc adm policy add-scc-to-group
# Priorità: più alto = preferita rispetto ad altre SCC candidate
priority: 10
# Privilegi container
allowPrivilegedContainer: false # ← mai consentire container privilegiati
allowPrivilegeEscalation: false # ← blocca sudo e setuid
# Capabilities Linux
defaultAddCapabilities: [] # ← nessuna capability aggiunta di default
requiredDropCapabilities:
  - ALL # ← dropare tutte le capabilities
allowedCapabilities: [] # ← nessuna capability aggiuntiva
consentita
# Strategia UID: deve essere esattamente 1001
runAsUser:
  type: MustRunAs # ← forza il valore specificato
  uid: 1001 # ← UID fisso per l'applicazione legacy
# Strategia fsGroup
fsGroup:
  type: MustRunAs
  ranges:
    - min: 1001
      max: 1001
# SELinux: usa il contesto MCS automatico del nodo
seLinuxContext:
  type: MustRunAs
# Supplemental groups
supplementalGroups:
  type: MustRunAs
  ranges:
    - min: 1001
      max: 1001
# Volumi consentiti
volumes:
  - configMap
  - downwardAPI
  - emptyDir
  - persistentVolumeClaim
  - projected
  - secret

```

```
# Rete: non può usare il network namespace del nodo
allowHostDirVolumePlugin: false
allowHostIPC: false
allowHostNetwork: false
allowHostPID: false
allowHostPorts: false
# Filesystem: lettura/scrittura normale (non read-only obbligatorio)
readOnlyRootFilesystem: false
```

Note Specifiche per Azure ARO

SCC su ARO: **restricted-v2** come Default

Su ARO (OpenShift 4.11+), la SCC **restricted-v2** è applicata di default a tutti i Pod che non hanno SCC specifiche assegnate. Questo è più restrittivo rispetto alle versioni precedenti di OpenShift che usavano **restricted**:

```
# Verificare la SCC applicata ai Pod nel namespace di sistema
oc get pods -n openshift-monitoring \
  -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}\n'
{.metadata.annotations.openshift\.io/scc}{"\n"}{end}'

# Output atteso (selezione):
# alertmanager-main-0      restricted-v2
# prometheus-k8s-0        restricted-v2
# thanos-querier-xxx      restricted-v2
```

Workload AKS su ARO: Possibili Problemi SCC

Un workload che funziona su **Azure Kubernetes Service (AKS)** potrebbe fallire su ARO per via delle SCC. AKS usa **Pod Security Admission (PSA)** invece di SCC, e alcuni container che girano su AKS come root potrebbero non rispettare i vincoli di **restricted-v2**:

```
# Diagnosticare un Pod che non parte su ARO (migrato da AKS)
oc describe pod migrated-app-xxx -n myproject | grep -E "Warning|Error|SCC"

# Soluzione 1: modificare l'immagine per girare come non-root
# Soluzione 2: creare una SCC custom intermedia (non anyuid se evitabile)
# Soluzione 3 (ultima risorsa): oc adm policy add-scc-to-user anyuid <SA>

# Verificare se un Deployment è compatibile con restricted-v2 PRIMA di deployarlo
oc adm policy scc-subject-review \
  -f deployment-from-aks.yaml \
  --serviceaccount myapp-sa \
  -n myproject
```

⚠ Attenzione: Evitare di assegnare la SCC **anyuid** o **privileged** senza analisi approfondita. Su ARO queste SCC aprono rischi di escalation dei privilegi. Preferire SCC custom con il minimo UID necessario.

ServiceAccount con Workload Identity su ARO

Su ARO con **Azure Workload Identity**, il ServiceAccount ha annotazioni speciali che collegano l'identità del Pod a una Managed Identity Azure:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: workload-identity-sa
  namespace: myproject
  annotations:
    azure.workload.identity/client-id: "12345678-abcd-ef01-2345-6789abcdef01" # ←
Client ID della Managed Identity
  labels:
    azure.workload.identity/use: "true" # ← abilita Workload Identity per questo
SA
automountServiceAccountToken: true # ← necessario per Workload Identity
(token OIDC proiettato)
```

Riepilogo dei Punti Chiave

- Ogni Pod in OpenShift gira sotto l'identità di un **ServiceAccount**; usare SA dedicati per ogni applicazione invece del SA **default**
 - Disabilitare **automountServiceAccountToken: false** quando il Pod non interagisce con l'API Kubernetes, per ridurre la superficie di attacco
 - Le **SCC** sono oggetti OpenShift cluster-scoped che controllano i privilegi dei container (UID, capabilities, volumi, SELinux); non esistono su Kubernetes upstream
 - La SCC **restricted-v2** è il default su ARO 4.11+: blocca container root, capability extra e volume host
 - OpenShift seleziona la SCC candidata più restrittiva compatibile con il security context del Pod; se nessuna SCC valida, il Pod viene rifiutato
 - Per workload legacy che richiedono UID specifici, creare SCC custom invece di assegnare **anyuid**
 - Su ARO con Workload Identity, il SA ha annotazioni Azure speciali che collegano il Pod a una Managed Identity Azure
-

Domande di Verifica

1. Un Pod non specifica `serviceAccountName`. Quale ServiceAccount viene usato?

- A) `cluster-admin`
- B) `openshift-default`
- C) `default` nel namespace del Pod ☒
- D) Nessuno — il Pod non viene creato

2. Quale SCC è applicata di default ai Pod su ARO 4.11+?

- A) `restricted`
- B) `restricted-v2` ☒
- C) `nonroot`
- D) `anyuid`

3. Un Pod fallisce con l'errore `forbidden: unable to validate against any security context constraint`. Quale è la causa più probabile?

- A) Il namespace non ha una ResourceQuota configurata
- B) Il Pod richiede un volume di tipo `hostPath` che non è consentito da nessuna SCC assegnata al suo SA ☒
- C) Il ServiceAccount non ha il ruolo `edit` nel namespace
- D) L'immagine container non è presente nel registry interno

4. Quale comando assegna la SCC `anyuid` al ServiceAccount `legacy-sa` nel namespace `legacy-project`?

- A) `oc adm policy add-scc-to-user anyuid legacy-sa -n legacy-project`
- B) `oc adm policy add-scc-to-user anyuid system:serviceaccount:legacy-project:legacy-sa` ☒
- C) `oc create rolebinding anyuid-binding --clusterrole=anyuid --serviceaccount=legacy-project:legacy-sa`
- D) `oc patch scc anyuid --type=json -p='[{"op":"add","path":"/users/-","value":"legacy-sa"}]'`

5. Qual è la best practice per un Pod applicativo che non interagisce con l'API Kubernetes?

- A) Usare il ServiceAccount `cluster-admin` per garantire accesso massimo
- B) Non specificare `serviceAccountName` per usare il SA `default`
- C) Creare un SA dedicato con `automountServiceAccountToken: false` ☒
- D) Montare il token manualmente come ConfigMap invece che come ServiceAccount